```sql
USE city_library;
GO

BEGIN TRY

    BEGIN TRANSACTION;

    INSERT INTO loans
    (
        book_id,
        member_id,
        loan_date,
        due_date,
        status
    )
    VALUES
    (
        1,
        3,
        CAST(GETDATE() AS DATE),
        DATEADD(DAY, 14, CAST(GETDATE() AS DATE)),
        'Active'
    );

    UPDATE books
    SET available_copies = available_copies - 1
    WHERE book_id = 1;

    COMMIT TRANSACTION;

    PRINT 'Borrow transaction completed successfully.';

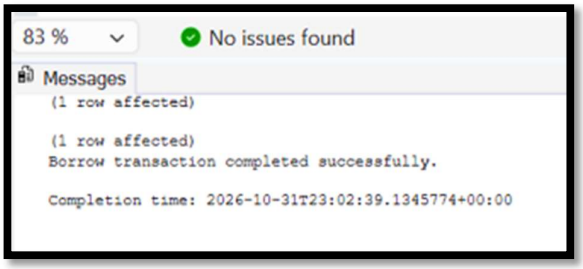
END TRY

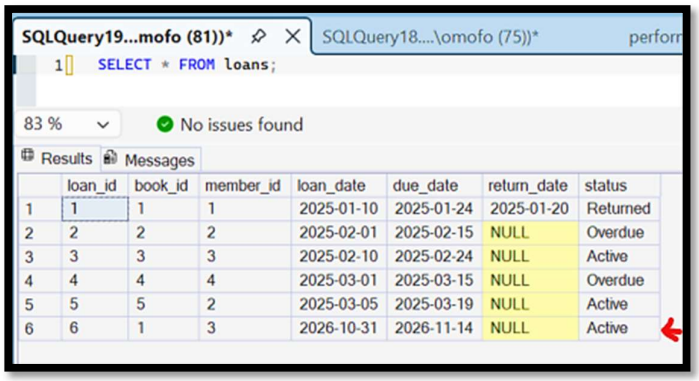
BEGIN CATCH

    ROLLBACK TRANSACTION;

    PRINT 'Transaction failed and was rolled back.';
    PRINT ERROR_MESSAGE();

END CATCH;
GO
```





	loan_id	book_id	member_id	loan_date	due_date	return_date	status
1	1	1	1	2025-01-10	2025-01-24	2025-01-20	Returned
2	2	2	2	2025-02-01	2025-02-15	NULL	Overdue
3	3	3	3	2025-02-10	2025-02-24	NULL	Active
4	4	4	4	2025-03-01	2025-03-15	NULL	Overdue
5	5	5	2	2025-03-05	2025-03-19	NULL	Active
6	6	1	3	2026-10-31	2026-11-14	NULL	Active

Transaction Management

To demonstrate transaction management in SQL Server, two borrowing scenarios were tested: a successful transaction and a failed transaction.

Successful Transaction: When a member borrows a book, a new record is added to the loans table and the number of available copies in the books table is reduced by one. Both operations are executed within a single transaction. The transaction is committed only after both statements complete successfully, ensuring the database remains consistent.

Outcome: These tests demonstrate the ACID properties of transactions, particularly

Atomicity and **Consistency**. Either all changes are committed successfully, or none of them are applied. This prevents situations where a book's available copies are reduced without a valid loan record being created.

```sql
USE city_library;
GO

BEGIN TRY

    BEGIN TRANSACTION;

    INSERT INTO loans
    (
        book_id,
        member_id,
        loan_date,
        due_date,
        status
    )
    VALUES
    (
        1,
        8,
        CAST(GETDATE() AS DATE),
        DATEADD(DAY, 14, CAST(GETDATE() AS DATE)),
        'Active'
    );

    UPDATE books
    SET available_copies = available_copies - 1
    WHERE book_id = 1;

    COMMIT TRANSACTION;

    PRINT 'Borrow transaction completed successfully.';

END TRY

BEGIN CATCH

    ROLLBACK TRANSACTION;

    PRINT 'Transaction failed and was rolled back.';
    PRINT ERROR_MESSAGE();

END CATCH;
GO
```

83 % — No issues found

Messages

```
(0 rows affected)
Transaction failed and was rolled back.
The INSERT statement conflicted with the FOREIGN KEY constraint "FK_Loans_Members". The conflict occurred in database "city_library", table "dbo.members", column 'member_id'.

Completion time: 2026-10-31T23:10:26.2574388+00:00
```

```sql
SELECT * FROM loans;
```

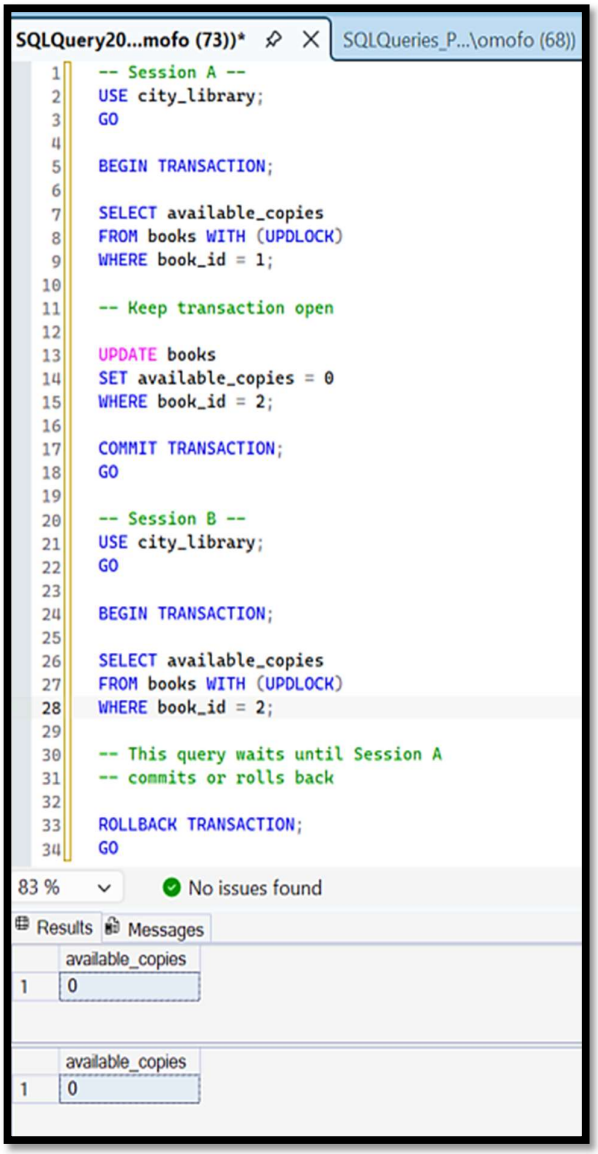
83 % — No issues found

Results | Messages

	loan_id	book_id	member_id	loan_date	due_date	return_date	status
1	1	1	1	2025-01-10	2025-01-24	2025-01-20	Returned
2	2	2	2	2025-02-01	2025-02-15	NULL	Overdue
3	3	3	3	2025-02-10	2025-02-24	NULL	Active
4	4	4	4	2025-03-01	2025-03-15	NULL	Overdue
5	5	5	2	2025-03-05	2025-03-19	NULL	Active
6	6	1	3	2026-10-31	2026-11-14	NULL	Active

Query Analysis
Failed Transaction: A second test was performed to demonstrate rollback functionality. The system attempted to create a loan using a non-existent member_id. This violated the foreign key constraint on the loans table and caused the transaction to fail. As a result, the transaction was rolled back and the update to available_copies was undone automatically.

CONCURRENCY CONTROL TEST



A concurrency test was conducted using two separate SQL Server sessions accessing the same book record.

In Session A, a transaction was started and an update lock was placed on the book with book_id = 1. The transaction was intentionally left open to hold the lock.

In Session B, another transaction attempted to access the same record using the same locking method. SQL Server blocked Session B because Session A was already holding a lock on the row.

After confirming the blocking behaviour, Session A updated the number of available copies and committed the transaction. Once the lock was released, Session B continued execution and was then rolled back.

This test demonstrates how SQL Server uses locking to prevent conflicting updates and maintain data consistency when multiple users access the same data simultaneously.